

Mission Control — Morning Brief

Build Specification v2

Document	Mission_Control_Morning_Brief_Spec_v2
Status	Authoritative build spec — Phase 1
Owner	Mike (GT40 Marine / NoodleBomb)
Date	2026-06-10
Layout contract	Morning_Brief_Prototype.html (ships alongside this spec)
Boundary suite	t5-agent-boundary.spec.ts (ships alongside this spec)

1. Purpose

Every morning at 3:00 AM, Mission Control collects the previous day’s sales data from both Shopify stores (GT40 Marine and NoodleBomb), computes trends, evaluates alert rules, asks Claude for a short reasoning pass over the numbers, and renders a single self-contained HTML brief to `briefs\`. The brief is the first thing read each morning: what happened yesterday, whether anything needs attention, and what to do about it.

Phase 1 is **Shopify-only and fully local** — no cloud services other than the Shopify Admin API and the Claude API. Everything runs on the Windows workstation under Task Scheduler. The OpenClaw agents have a strictly limited relationship to this system (section 9).

2. Scope & Rollout

Week	Sources	State
1 (this spec)	Shopify: GT40 Marine + NoodleBomb	Build everything in this document
2	Amazon SP-API	Credentials staged in <code>secrets.json</code> now (from existing user-scope env vars); collector built later
3	eBay, GA4	Blank/disabled entries in config now; nothing else

Out of scope for Phase 1: inventory-level alerting, ad-spend ingestion, email/SMS delivery of the brief, historical backfill beyond 35 days.

3. Architecture & Data Flow

```
Task Scheduler (3:00 AM, -WakeToRun)
└─ run_morning_brief.ps1      (orchestrator, exit code = health)
    └─ Load-Secrets.ps1      (single source of credentials)
    └─ collect_shopify.ps1    (GT40 + NoodleBomb → data\YYYY-MM-DD\*.json)
    └─ generate_brief.py
        └─ trend math         (section 7.2)
        └─ alert engine       (alert_rules.json, section 7.3)
        └─ Claude reasoning pass (section 8; falls back per 8.4)
        └─ HTML renderer      (must match Morning_Brief_Prototype.html)
            └─ briefs\brief_YYYY-MM-DD.html
```

Every step appends to `logs\mission_control_YYYY-MM-DD.log` through the shared logging helper (section 4.2). A step that fails must say so in the log and propagate a nonzero exit code to the orchestrator; the orchestrator must still attempt the brief with whatever data exists (a degraded brief with a visible failure banner beats no brief).

All date logic uses **America/Los_Angeles**. "Yesterday" means the previous calendar day in that timezone. Currency is USD.

4. Directory Tree

Root: `C:\Users\12534\N00DLBOMB_ARCHIVE\mission_control\`

```

mission_control\
├─ config\
│   ├─ secrets.template.json    (committed reference; placeholders only)
│   ├─ secrets.json             (NEVER committed; icacIs-locked, section 5.3)
│   ├─ sources.json             (which sources are enabled, store metadata)
│   └─ alert_rules.json         (alert engine rules, section 7.3)
├─ scripts\
│   ├─ Load-Secrets.ps1        (credential loader, section 5.2)
│   ├─ Write-MCLog.ps1          (logging helper, section 4.2)
│   ├─ collect_shopify.ps1      (section 6)
│   ├─ stub_heartbeat.ps1       (day-one scheduler stub, section 10.2)
│   └─ run_morning_brief.ps1    (orchestrator, section 10.1)
├─ engine\
│   ├─ generate_brief.py         (section 7 + 8)
│   └─ templates\
│       └─ claude_prompt.txt     (section 8.2)
├─ data\
│   └─ YYYY-MM-DD\
│       ├─ shopify_gt40.json
│       └─ shopify_noodlebomb.json
├─ briefs\
│   └─ brief_YYYY-MM-DD.html
└─ logs\
    ├─ mission_control_YYYY-MM-DD.log
    └─ scheduler_heartbeat.txt   (written by the stub; kept afterward as a canary)

```

Rules:

- **4.1** Nothing inside `mission_control\` may reference `active_tasks.json`, `runtime_tick.ps1`, `MASTER_INDEX.md`, or any other OpenClaw runtime file. This is enforced by acceptance test T5 (section 11) and is non-negotiable.
- **4.2 Logging helper.** `Write-MCLog.ps1` exposes `Write-MCLog -Level INFO|WARN|ERROR -Step <name> -Message <text>`. Format: `2026-06-10T03:00:12-07:00 [INFO ] [collect_shopify] message`. It appends to the dated log file, creating it (and `logs\`) if missing. `generate_brief.py` writes the same format directly.
- **4.3** `data\` and `briefs\` are retained 90 days; the orchestrator prunes older folders/files at the end of each run.

5. Configuration & Secrets

5.1 Files

`sources.json` (committed-safe, no secrets):

```
{
  "timezone": "America/Los_Angeles",
  "currency": "USD",
  "sources": {
    "shopify_gt40": { "enabled": true, "label": "GT40 Marine", "kind": "shopify" },
    "shopify_noodlebomb": { "enabled": true, "label": "NoodleBomb", "kind": "shopify" },
    "amazon_sp_api": { "enabled": false, "label": "Amazon", "kind": "amazon",
    "note": "week 2" },
    "ebay": { "enabled": false, "label": "eBay", "kind": "ebay",
    "note": "week 3" },
    "ga4": { "enabled": false, "label": "GA4", "kind": "ga4",
    "note": "week 3" }
  },
  "claude": { "model": "claude-opus-4-8", "max_tokens": 2000, "timeout_seconds": 120 }
}
```

`secrets.template.json` defines the exact shape of `secrets.json`. Every leaf value in the template is the literal string `"REPLACE_ME"` (or `" "` for the week-2/3 sources). `secrets.json` is created by copying the template and filling values. The Amazon SP-API values are **migrated from the existing user-scope environment variables** — read once with `[Environment]::GetEnvironmentVariable($name, 'User')` and written into `secrets.json`; do not re-provision them and do not create new env vars.

5.2 Load-Secrets.ps1 — the only credential path

All PowerShell credential access goes through `Load-Secrets.ps1`. No script may read tokens from environment variables, inline literals, or anywhere else. The loader must:

1. Read `config\secrets.json`; **throw** (terminating error) if the file is missing or invalid JSON.
2. Validate the key structure against `secrets.template.json`; **throw** listing the missing keys if any required Shopify or Anthropic key is absent or still `"REPLACE_ME"`. Disabled sources (per `sources.json`) are exempt from the filled-value check.
3. Return the parsed object; expose `Get-MCSecret -Secrets $s -Path "shopify.gt40.admin_api_token"` for dotted-path access.
4. Never write any secret value to the log, console, or an error message — key *names* are loggable, values never.

`generate_brief.py` reads `config\secrets.json` directly (same machine, same user, ACL-protected) and applies rules 1, 2, and 4 identically.

5.3 Locking secrets.json

Immediately after `secrets.json` is first filled:

```
icacls "C:\Users\12534\N00DLEBOMB_ARCHIVE\mission_control\config\secrets.json" /inheritance:r /grant:r "12534:F"
```

Result: only user `12534` (and processes running as that user, which includes the scheduled task) can read it. T1 verifies the ACL.

5.4 Credential inventory (Phase 1)

Path in secrets.json	Source	When
<code>anthropic.api_key</code>	Anthropic Console	task 4
<code>shopify.gt40.store_domain</code> , <code>.admin_api_token</code>	Custom app, created by Mike (see 6.1)	task 2 — builder stops and waits
<code>shopify.noodlebomb.store_domain</code> , <code>.admin_api_token</code>	Custom app, created by Mike (see 6.1)	task 2 — builder stops and waits
<code>amazon_sp_api.*</code>	Existing <code>-s</code> user env vars, migrated	task 1
<code>ebay.*</code> , <code>ga4.*</code>	—	blank until week 3

6. Data Collection — collect_shopify.ps1

6.1 Credentials

One **custom app** per store (Shopify Admin → Settings → Apps and sales channels → Develop apps), Admin API scopes: `read_orders` , `read_products` . The builder must stop at the start of task 2, hand Mike the exact click-path instructions for both stores, and wait for the tokens before continuing. Tokens land in `secrets.json` only.

6.2 Behavior

For each enabled Shopify source, the collector:

1. Calls Shopify Admin REST API `GET /admin/api/2026-01/orders.json` with `status=any` , `created_at_min` / `created_at_max` spanning the **35 days ending yesterday 23:59:59**

(America/Los_Angeles, converted to ISO-8601 with offset), paginating via the `Link: rel="next"` header, `limit=250`.

2. Retries each HTTP call up to **3 times with backoff 2s/4s/8s** on 429 and 5xx; respects `Retry-After` when present.
3. Reduces orders to one daily-rollup record per calendar day (section 6.3) and writes `data\YYYY-MM-DD\shopify_<store>.json` (the folder is named for **yesterday**, the day the brief covers).
4. On unrecoverable failure: writes `data\YYYY-MM-DD\shopify_<store>.FAILED.json` containing `{ "error": "<message>", "at": "<iso>" }`, logs ERROR, and exits nonzero. It must not leave a partial/invalid data file.

Cancelled orders (`cancelled_at != null`) and test orders (`test: true`) are excluded. Revenue per order = `current_total_price` (post-refund, pre-shipping-tax adjustments are acceptable for Phase 1).

6.3 Data file schema

```
{
  "source": "shopify_gt40",
  "label": "GT40 Marine",
  "generated_at": "2026-06-11T03:01:42-07:00",
  "window": { "start": "2026-05-07", "end": "2026-06-10" },
  "days": [
    { "date": "2026-06-10", "revenue": 1234.56, "orders": 7, "units": 11 }
  ]
}
```

`days` covers every date in the window, **including zero days** (`revenue: 0`, `orders: 0`, `units: 0`) so the trend math never has gaps.

7. Brief Generation — `generate_brief.py`

Invocation: `python engine\generate_brief.py --date YYYY-MM-DD [--no-clause] ( --date defaults to yesterday; --no-clause forces the fallback path for testing — used by T4).`

7.1 Inputs

- `data\<date>\shopify_*.json` (and `*.FAILED.json` markers)
- `config\sources.json`, `config\alert_rules.json`, `config\secrets.json`

A missing or FAILED data file for an enabled source does **not** abort the run: the brief renders with that source's panel in the `failed` state and the `collection_failure` alert fires.

7.2 Trend math (exact definitions)

For each store, and for the combined total, with D = the brief date:

Metric	Definition
revenue , orders , units	The day- D values from the data file
ao _v	$\text{revenue} / \text{orders}$; null when $\text{orders} == 0$
avg7 (per metric)	Mean of the metric over $D-7 \dots D-1$ (7 days)
avg28 (per metric)	Mean over $D-28 \dots D-1$ (28 days)
delta7_pct	$(\text{value} - \text{avg7}) / \text{avg7} * 100$; null when $\text{avg7} == 0$
delta28_pct	same vs avg28
wow_pct	vs the same weekday one week earlier ($D-7$'s value); null when that value is 0
spark14	The raw daily revenue values for $D-13 \dots D$ (14 numbers, oldest first) — feeds the prototype's spark bars

All means use the calendar days present in the data window; missing days were zero-filled by the collector, so no imputation happens here. Round percentages to 1 decimal for display; keep full precision in the JSON handed to Claude.

7.3 Alert engine — alert_rules.json

```
{
  "rules": [
    { "id": "revenue_drop_30_vs_avg7", "enabled": true, "scope": "per_store",
      "metric": "revenue", "baseline": "avg7", "comparator": "delta_pct_lte", "threshold": -30,
      "severity": "critical", "message": "{label} revenue ${value} is {delta}% vs 7-day avg ${baseline}" },
    { "id": "zero_orders", "enabled": true, "scope": "per_store",
      "metric": "orders", "baseline": "absolute", "comparator": "lte", "threshold": 0,
      "severity": "warn", "message": "{label} had zero orders yesterday" },
    { "id": "aov_swing_25", "enabled": true, "scope": "per_store",
      "metric": "aov", "baseline": "avg28", "comparator": "abs_delta_pct_gte", "threshold": 25,
      "severity": "warn", "message": "{label} AOV ${value} moved {delta}% vs 28-day avg ${baseline}" },
    { "id": "revenue_spike_50_vs_avg7", "enabled": true, "scope": "per_store",
      "metric": "revenue", "baseline": "avg7", "comparator": "delta_pct_gte", "threshold": 50,
      "severity": "info", "message": "{label} revenue ${value} is +{delta}% vs 7-day avg – find out why and do it again" },
    { "id": "collection_failure", "enabled": true, "scope": "per_store",
      "metric": "_collection", "baseline": "absolute", "comparator": "failed", "threshold": null,
      "severity": "critical", "message": "{label}: data collection FAILED – brief is incomplete for this source" }
  ]
}
```

Engine semantics:

- scope: "per_store" evaluates the rule once per enabled source; "combined" (legal value, unused by defaults) evaluates against totals.
- Comparators: lte / gte test the raw metric value against threshold ;
delta_pct_lte / delta_pct_gte test the signed delta vs the named baseline; abs_delta_pct_gte tests the absolute delta; failed fires when the source's data file is missing or FAILED.
- A rule whose inputs are null (e.g. AOV with zero orders, delta with a zero baseline) does **not** fire and logs at DEBUG level why.
- Unknown rule fields are ignored; an unknown comparator or metric is a config error → log ERROR and treat the rule as disabled (never crash the brief over a bad rule).
- Fired alerts sort: critical, warn, info; then rule order.

7.4 HTML renderer

The renderer must reproduce `Morning_Brief_Prototype.html` **exactly** in structure: same element hierarchy, same `id` attributes, same class names, same CSS custom properties (design tokens), same fonts. The prototype's `<style>` block is copied verbatim into each brief (the brief must be a single self-contained file with zero external requests). Dynamic content fills the elements carrying `data-` field attributes; repeated structures (store panels, alert cards, insight items, spark bars) clone the prototype's corresponding element as the template. The `data-mode` attribute on `<body>` is `ai` or `fallback` (section 8.4); per-store panels carry `data-state="ok|failed"`.

Output: `briefs\brief_YYYY-MM-DD.html`, UTF-8, atomic write (write `*.tmp`, then rename).

8. Claude Reasoning Pass

8.1 Call

- Python, official anthropic SDK (`pip install anthropic`).
- Model: **claude-opus-4-8** (from `sources.json`, never hard-coded).
- `max_tokens`: 2000, default thinking (omit the parameter), one call per brief, no retry loops beyond the SDK's built-in retries, hard timeout 120 s.
- Structured output via `output_config.format` with the JSON schema in 8.3 — the response is machine-parsed, never scraped from prose.

8.2 Prompt template — engine\templates\claude_prompt.txt

You are the analyst writing the 3:00 AM morning brief for a two-store e-commerce operation: GT40 Marine (marine parts) and NoodleBomb (ramen sauces and seasonings). Your reader is the owner, drinking his first coffee. He already sees the numbers — your job is the "so what".

Today's date: {{DATE}}

Yesterday's metrics and trend math (full precision):
{{METRICS_JSON}}

Alerts fired by the rule engine:
{{ALERTS_JSON}}

Write:

- headline: one sentence, the single most important thing about yesterday.
- insights: 3 to 5 bullets. Each must reference a specific number from the data. Call out divergence between the two stores when present. Never invent numbers that are not in the data.
- actions: 1 to 3 concrete things worth doing today, ordered by impact. If nothing needs doing, say so in one action ("No action needed — ...").

Tone: direct, specific, zero filler. If the data is partial (a collection failure is flagged in the alerts), say so plainly and reason only from what's present.

{{METRICS_JSON}} is the full trend-math object (7.2) for both stores plus combined; {{ALERTS_JSON}} is the fired-alert list (empty array when quiet).

8.3 Response schema

```
{
  "type": "object",
  "properties": {
    "headline": { "type": "string" },
    "insights": { "type": "array", "items": { "type": "string" } },
    "actions": { "type": "array", "items": { "type": "string" } }
  },
  "required": ["headline", "insights", "actions"],
  "additionalProperties": false
}
```

8.4 Fallback (acceptance test T4)

The brief must **never** fail because the Claude API is unreachable. On `anthropic.APIConnectionError`, `anthropic.APIStatusError`, `timeout`, a missing/placeholder API key, or `--no-claude`:

1. Log `WARN` with the exception class (never the key).
2. Generate a **deterministic** insights block from the trend math alone:
 - headline: the largest-magnitude `delta7_pct` across stores, phrased mechanically (e.g. "NoodleBomb revenue -42.3% vs 7-day average").
 - insights: one line per store (revenue, orders, AOV vs avg7) plus one line per fired alert.
 - actions: ["Review the numbers manually – AI reasoning was unavailable for this brief."]
3. Render with `data-mode="fallback"` so the brief visibly carries the "FALLBACK — no AI pass" badge from the prototype.
4. Exit 0. The fallback path is a success path.

9. Agent Boundary Requirements (non-negotiable)

The OpenClaw gateway (dashboard on **port 18789**) runs two agents, Chili and Buster. Their relationship to `mission_control`:

	Chili	Buster
Awareness of <code>mission_control</code>	ZERO	Operational, read-mostly
<code>MASTER_INDEX.md</code> entry	must not exist	n/a
<code>active_tasks.json</code> entries	must not exist	n/a
Heartbeat read path	must not touch <code>mission_control</code>	n/a
Execute	—	<code>run_morning_brief.ps1 -Force</code> only
Read	—	<code>briefs\</code> and <code>logs\</code> only
Write to <code>config\</code>	never	never
Grant scoping	—	agent-scoped (<code>agentId: "buster"</code>), never a wildcard agent or wildcard path

And the inverse (rule 4.1): nothing inside `mission_control\` references `active_tasks.json`, `runtime_tick.ps1`, or any OpenClaw runtime file.

Enforcement is `t5-agent-boundary.spec.ts`, run against the **live** gateway. A gateway that is down or reports a degraded permissions snapshot **fails the suite loudly** — it must never pass green by skipping.

10. Orchestration & Scheduling

10.1 run_morning_brief.ps1

```
param([switch]$Force)
```

1. Re-entrancy guard: if today's brief already exists and `-Force` is not set, log INFO and exit 0.
2. Load `Secrets.ps1` (throws → exit 2, nothing else runs).
3. `collect_shopify.ps1` (its nonzero exit is recorded but does not abort).
4. `python engine\generate_brief.py --date <yesterday>` (nonzero → exit 3).
5. Prune `data\ / briefs\` older than 90 days; log run summary (duration, alerts fired, mode ai/fallback).
6. Exit 0 when a brief was produced (even degraded/fallback); the exit code is the scheduled task's health signal.

10.2 Task Scheduler — registered on day one

Task name: `MissionControl Morning Brief`, registered immediately at the start of the build with the **stub** action, then updated in place when the orchestrator lands (same task, `Set-ScheduledTask` swapping the action — never a second task).

- **Stub:** `scripts\stub_heartbeat.ps1` — appends one ISO-8601 line to `logs\scheduler_heartbeat.txt` and exits 0. Purpose: burn down the wake-from-sleep / scheduling risk on night one while the collectors are still being built.
- **Trigger:** daily 3:00 AM local.
- **Settings:** `-WakeToRun`, `StartWhenAvailable` (catch-up if the machine was off at 3:00), `-AllowStartIfOnBatteries`, `-DontStopIfGoingOnBatteries`, execution time limit 1 hour, no parallel instances (`-MultipleInstances IgnoreNew`).
- **Action (final):** `powershell.exe -NoProfile -ExecutionPolicy Bypass -File "... \scripts\run_morning_brief.ps1"`.
- **Principal:** user 12534, run whether logged on or not.

10.3 Wake-from-sleep verification

Morning after registration: `scheduler_heartbeat.txt` contains a ~3:00 AM timestamp **and** `(Get-ScheduledTaskInfo "MissionControl Morning Brief").LastTaskResult -eq 0`. If the machine slept through it, fix power settings (`powercfg /waketimers`, BIOS wake, "Allow wake timers") before building further — this is exactly the risk the day-one stub exists to surface.

11. Acceptance Tests (T1–T6)

All must pass before the ship gate. T1–T4 and T6 are PowerShell/pytest-style checks run locally; T5 is the Playwright suite.

#	Name	Pass criteria
T1	Scaffold & config	Full tree from section 4 exists. <code>Load-Secrets.ps1</code> throws on missing file, invalid JSON, and <code>REPLACE_ME</code> values (each case tested). <code>sources.json</code> and <code>alert_rules.json</code> parse. <code>icacls</code> output for <code>secrets.json</code> shows inheritance removed and only <code>12534:F</code> .
T2	Collectors	With real tokens: both stores produce schema-valid data files covering 35 zero-filled days. With an invalid token (test): 3 retries observed in the log, <code>*.FAILED.json</code> written, nonzero exit, no partial data file.
T3	Trend math & renderer	Against the committed fixture dataset: every value in 7.2 matches hand-computed expectations to 2 decimals; rendered HTML contains the prototype's exact <code>id</code> set, design-token block, and <code>data-field</code> population; zero external resource references.
T4	AI fallback	<code>generate_brief.py --no-claude</code> (and separately, with an unreachable API) produces a complete brief with <code>data-mode="fallback"</code> , the fallback badge visible, exit 0, WARN in the log.
T5	Agent boundary	<code>t5-agent-boundary.spec.ts</code> green against the running gateway on 18789. The suite itself must fail (not skip) when the gateway is down or degraded. Includes the <code>patternReaches()</code> adversarial unit fixtures — all green.
T6	End-to-end & scheduling	<code>run_morning_brief.ps1 -Force</code> produces today's brief end-to-end with exit 0 and a complete log trail. Scheduled task exists, <code>NextRunTime</code> is 3:00 AM, <code>LastTaskResult</code> 0, and <code>scheduler_heartbeat.txt</code> shows at least one overnight 3 AM line.

Ship gate: tasks 1–5 built + T1–T6 green + boundary suite green = live Shopify-only brief. **Stop and report** before touching Amazon (week 2) or eBay/GA4 (week 3).